TESTING GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025 **Audience**: QA Engineers, Test Leads, Developers

---

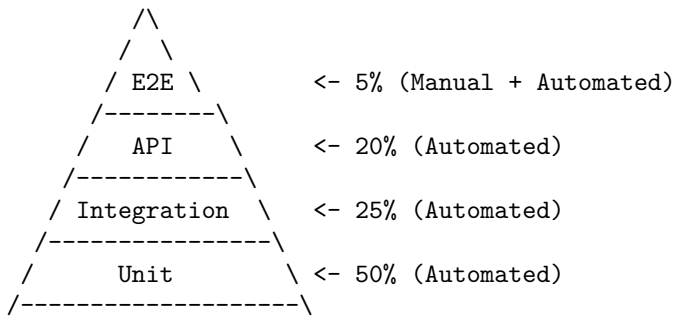
TABLE OF CONTENTS

1. Testing Strategy Overview
2. Test Environment Setup
3. Unit Testing
4. Integration Testing
5. API Testing
6. Performance Testing
7. Security Testing
8. ML Model Testing
9. Edge Device Testing
10. User Acceptance Testing
11. Test Automation
12. Defect Management

---

1. TESTING STRATEGY OVERVIEW

1.1 Testing Pyramid



1.2 Testing Phases

Phase	Type	Scope	Owner
Development	Unit Tests	Functions, Classes	Developers
Development	Integration Tests	Services, APIs	Developers
CI Pipeline	Automated Tests	All Unit + Integration	DevOps
Pre-Release	API Tests	All Endpoints	QA
Pre-Release	Performance Tests	Load, Stress	QA
Pre-Release	Security Tests	OWASP, Penetration	Security
Pre-Release	ML Validation	Model Performance	ML Engineers
Release	UAT	Business Flows	Business Users
Release	Edge Tests	Device Integration	Hardware QA

1.3 Test Categories

Category	Description	Coverage Target
Functional	Features work as specified	100% of requirements
Performance	Response times, throughput	<500ms API, >1000 TPS
Security	Auth, injection, data protection	OWASP Top 10
Reliability	Error handling, recovery	99.9% uptime
ML Accuracy	Model predictions	>95% accuracy, <5% false positive
Usability	User interface, workflows	SUS score >70

1.4 Entry/Exit Criteria

Entry Criteria (Start Testing)

- Code review completed
- Unit tests passing (>80% coverage)
- Build successful
- Test environment available
- Test data prepared

Exit Criteria (Release)

- All critical/high defects resolved
- Test case pass rate >95%
- Performance benchmarks met
- Security scan passed
- UAT sign-off received

---

2. TEST ENVIRONMENT SETUP

2.1 Environment Overview

Environment	Purpose	Data	Access
Local	Development	Mock/Synthetic	Developer machine
Dev	Integration	Anonymized subset	VPN
QA	Testing	Anonymized full set	VPN
Staging	Pre-production	Production mirror	VPN + approval
Production	Live system	Real data	VPN + MFA

2.2 QA Environment Specs

```
QA Environment Infrastructure
==============================

Kubernetes Cluster:
- 2x Master Nodes
- 3x Worker Nodes (16 CPU, 64GB RAM each)
- 1x GPU Node (32GB GPU, for ML testing)

Databases:
```

```
- PostgreSQL 16 (single node, 8 CPU, 32GB RAM)
- InfluxDB 2.7 (single node, 8 CPU, 32GB RAM)
- Milvus 2.3 (single node, 8 CPU, 32GB RAM)
- Redis 7.2 (single node, 4 CPU, 16GB RAM)

External Services:
- Kafka (3 brokers, 4 CPU, 16GB RAM each)
- MinIO (2 nodes, 1TB storage each)

Network:
- Isolated VLAN
- VPN access required
- No external internet access (security)
```

2.3 Test Data Management

Data Categories

Type	Source	Anonymization	Refresh
Master Data	Production clone	PII removed	Weekly
Sensor Data	Synthetic generator	N/A	On-demand
Historical Data	Production sample	Masked	Monthly
Edge Cases	Manual creation	N/A	As needed

Synthetic Data Generator

```python
# tests/data_generator/sensor_data.py
import numpy as np
from datetime import datetime, timedelta
from typing import Generator, Dict
import json

class SensorDataGenerator:
    """Generate synthetic sensor data for testing"""

    def __init__(self, seed: int = 42):
        np.random.seed(seed)

    def generate_normal_reading(
        self,
        sensor_id: str,
        asset_id: str,
        base_value: float,
        noise_std: float = 0.1
    ) -> Dict:
        """Generate normal sensor reading"""
        return {
            "sensor_id": sensor_id,
            "asset_id": asset_id,
            "timestamp": datetime.utcnow().isoformat(),
            "value": base_value + np.random.normal(0, noise_std),
            "quality": 100,
            "unit": "unit"
```

```
        }

    def generate_anomaly_reading(
        self,
        sensor_id: str,
        asset_id: str,
        base_value: float,
        anomaly_factor: float = 3.0
    ) -> Dict:
        """Generate anomalous sensor reading"""
        anomaly_value = base_value * (1 + np.random.uniform(0.5, anomaly_factor))
        return {
            "sensor_id": sensor_id,
            "asset_id": asset_id,
            "timestamp": datetime.utcnow().isoformat(),
            "value": anomaly_value,
            "quality": 100,
            "unit": "unit"
        }

    def generate_degradation_series(
        self,
        sensor_id: str,
        asset_id: str,
        base_value: float,
        days: int,
        degradation_rate: float = 0.01
    ) -> Generator[Dict, None, None]:
        """Generate series showing gradual degradation"""
        start_time = datetime.utcnow() - timedelta(days=days)

        for day in range(days):
            current_time = start_time + timedelta(days=day)
            degraded_value = base_value * (1 + degradation_rate * day)

            # Add readings throughout the day
            for hour in range(0, 24, 4):
                timestamp = current_time + timedelta(hours=hour)
                yield {
                    "sensor_id": sensor_id,
                    "asset_id": asset_id,
                    "timestamp": timestamp.isoformat(),
                    "value": degraded_value + np.random.normal(0, 0.05),
                    "quality": 100,
                    "unit": "unit"
                }

# Usage
generator = SensorDataGenerator()

# Generate 100 normal readings
normal_readings = [
    generator.generate_normal_reading("SENS-001", "PUMP-001", 50.0)
    for _ in range(100)
```

```
]

# Generate degradation series
degradation_data = list(generator.generate_degradation_series(
    "SENS-001", "PUMP-001", 50.0, days=30, degradation_rate=0.02
))
```

Test Data Setup Script

```
#!/bin/bash
# tests/setup_test_data.sh

echo "Setting up test data for QA environment..."

# PostgreSQL test data
psql -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB << 'EOF'
-- Clear existing test data
DELETE FROM asset_mgmt.assets WHERE asset_tag LIKE 'TEST-%';
DELETE FROM asset_mgmt.locations WHERE location_id IN (SELECT location_id FROM asset_mgmt.locations WHE

-- Insert test locations
INSERT INTO asset_mgmt.locations (location_id, name, location_type, parent_location_id)
VALUES
    ('11111111-1111-1111-1111-111111111111', 'Test Refinery', 'refinery', NULL),
    ('22222222-2222-2222-2222-222222222222', 'Test Unit A', 'unit', '11111111-1111-1111-1111-1111111111
    ('33333333-3333-3333-3333-333333333333', 'Test Unit B', 'unit', '11111111-1111-1111-1111-1111111111

-- Insert test assets
INSERT INTO asset_mgmt.assets (asset_id, asset_tag, name, asset_type, location_id, status, criticality)
VALUES
    ('aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa', 'TEST-PMP-001', 'Test Pump 1', 'pump', '22222222-2222-2222
    ('bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb', 'TEST-PMP-002', 'Test Pump 2', 'pump', '22222222-2222-2222
    ('cccccccc-cccc-cccc-cccc-cccccccccccc', 'TEST-CMP-001', 'Test Compressor', 'compressor', '33333333

-- Insert test sensors
INSERT INTO asset_mgmt.sensors (sensor_id, asset_id, sensor_tag, sensor_type, unit, min_value, max_valu
VALUES
    ('s1111111-1111-1111-1111-111111111111', 'aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa', 'TEST-VIB-001', 'v
    ('s2222222-2222-2222-2222-222222222222', 'aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa', 'TEST-TEMP-001', '
    ('s3333333-3333-3333-3333-333333333333', 'bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb', 'TEST-VIB-002', 'v
EOF

# Generate synthetic sensor data and load to InfluxDB
python tests/data_generator/load_influx_data.py

echo "Test data setup complete!"
```

---

3. UNIT TESTING

3.1 Python Unit Testing (pytest)

Test Structure

```
tests/
  unit/
      __init__.py
      conftest.py              # Shared fixtures
      test_models/
          test_asset.py
          test_sensor.py
          test_alert.py
      test_services/
          test_asset_service.py
          test_alert_service.py
          test_prediction_service.py
      test_repositories/
          test_asset_repository.py
      test_utils/
           test_validators.py
           test_calculations.py
  integration/
  e2e/
```

Fixtures (conftest.py)

```python
# tests/unit/conftest.py
import pytest
from unittest.mock import AsyncMock, MagicMock
from uuid import uuid4
from datetime import datetime

from app.models.asset import Asset
from app.schemas.asset import AssetCreate, AssetResponse

@pytest.fixture
def sample_asset_id():
    return uuid4()

@pytest.fixture
def sample_location_id():
    return uuid4()

@pytest.fixture
def sample_asset_create(sample_location_id):
    return AssetCreate(
        asset_tag="TEST-PMP-001",
        name="Test Pump",
        asset_type="pump",
        location_id=sample_location_id,
        criticality="high"
    )
```

```python
@pytest.fixture
def sample_asset_model(sample_asset_id, sample_location_id):
    asset = MagicMock(spec=Asset)
    asset.asset_id = sample_asset_id
    asset.asset_tag = "TEST-PMP-001"
    asset.name = "Test Pump"
    asset.asset_type = "pump"
    asset.location_id = sample_location_id
    asset.status = "operational"
    asset.criticality = "high"
    asset.created_at = datetime.utcnow()
    asset.updated_at = datetime.utcnow()
    return asset

@pytest.fixture
def mock_db_session():
    return AsyncMock()

@pytest.fixture
def mock_repository(mock_db_session):
    repo = AsyncMock()
    repo.session = mock_db_session
    return repo

@pytest.fixture
def mock_influx_client():
    return AsyncMock()

@pytest.fixture
def mock_kafka_producer():
    return AsyncMock()
```

Service Unit Tests

```python
# tests/unit/test_services/test_asset_service.py
import pytest
from unittest.mock import AsyncMock, patch
from uuid import uuid4

from fastapi import HTTPException

from app.services.asset_service import AssetService
from app.schemas.asset import AssetCreate, AssetUpdate

class TestAssetService:
    """Unit tests for AssetService"""

    @pytest.fixture
    def service(self, mock_repository, mock_influx_client):
        return AssetService(mock_repository, mock_influx_client)

    # CREATE TESTS
    @pytest.mark.asyncio
    async def test_create_asset_success(
```

```
    self,
    service,
    mock_repository,
    sample_asset_create,
    sample_asset_model
):
    """Test successful asset creation"""
    # Arrange
    mock_repository.get_by_tag.return_value = None
    mock_repository.create.return_value = sample_asset_model

    # Act
    result = await service.create_asset(sample_asset_create)

    # Assert
    assert result.asset_tag == "TEST-PMP-001"
    assert result.name == "Test Pump"
    mock_repository.get_by_tag.assert_called_once_with("TEST-PMP-001")
    mock_repository.create.assert_called_once()

@pytest.mark.asyncio
async def test_create_asset_duplicate_tag(
    self,
    service,
    mock_repository,
    sample_asset_create,
    sample_asset_model
):
    """Test creating asset with duplicate tag raises error"""
    # Arrange
    mock_repository.get_by_tag.return_value = sample_asset_model

    # Act & Assert
    with pytest.raises(HTTPException) as exc_info:
        await service.create_asset(sample_asset_create)

    assert exc_info.value.status_code == 409
    assert "already exists" in str(exc_info.value.detail)

# GET TESTS
@pytest.mark.asyncio
async def test_get_asset_success(
    self,
    service,
    mock_repository,
    mock_influx_client,
    sample_asset_id,
    sample_asset_model
):
    """Test getting asset by ID"""
    # Arrange
    mock_repository.get.return_value = sample_asset_model
    mock_influx_client.query.return_value = [{"_value": 0.85}]
```

```
    # Act
    result = await service.get_asset(sample_asset_id)

    # Assert
    assert result.asset_id == sample_asset_id
    assert result.health_index == 0.85

@pytest.mark.asyncio
async def test_get_asset_not_found(
    self,
    service,
    mock_repository,
    sample_asset_id
):
    """Test getting non-existent asset raises 404"""
    # Arrange
    mock_repository.get.return_value = None

    # Act & Assert
    with pytest.raises(HTTPException) as exc_info:
        await service.get_asset(sample_asset_id)

    assert exc_info.value.status_code == 404

# UPDATE TESTS
@pytest.mark.asyncio
async def test_update_asset_success(
    self,
    service,
    mock_repository,
    sample_asset_id,
    sample_asset_model
):
    """Test updating asset"""
    # Arrange
    update_data = AssetUpdate(name="Updated Pump Name")
    sample_asset_model.name = "Updated Pump Name"
    mock_repository.update.return_value = sample_asset_model

    # Act
    result = await service.update_asset(sample_asset_id, update_data)

    # Assert
    assert result.name == "Updated Pump Name"

# DELETE TESTS
@pytest.mark.asyncio
async def test_delete_asset_success(
    self,
    service,
    mock_repository,
    sample_asset_id
):
    """Test deleting asset"""
```

```python
        # Arrange
        mock_repository.delete.return_value = True

        # Act
        await service.delete_asset(sample_asset_id)

        # Assert
        mock_repository.delete.assert_called_once_with(sample_asset_id)

    @pytest.mark.asyncio
    async def test_delete_asset_not_found(
        self,
        service,
        mock_repository,
        sample_asset_id
    ):
        """Test deleting non-existent asset raises 404"""
        # Arrange
        mock_repository.delete.return_value = False

        # Act & Assert
        with pytest.raises(HTTPException) as exc_info:
            await service.delete_asset(sample_asset_id)

        assert exc_info.value.status_code == 404
```

Running Unit Tests

```bash
# Run all unit tests
pytest tests/unit/ -v

# Run with coverage
pytest tests/unit/ --cov=app --cov-report=html --cov-report=term-missing

# Run specific test file
pytest tests/unit/test_services/test_asset_service.py -v

# Run specific test
pytest tests/unit/test_services/test_asset_service.py::TestAssetService::test_create_asset_success -v

# Run with markers
pytest tests/unit/ -m "not slow" -v

# Generate JUnit report
pytest tests/unit/ --junitxml=reports/unit-tests.xml
```

3.2 Frontend Unit Testing (Jest/Vitest)

```typescript
// tests/components/AssetCard.test.tsx
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, vi } from 'vitest'
import { AssetCard } from '@/components/assets/AssetCard'

describe('AssetCard', () => {
```

```
  const mockAsset = {
    asset_id: '123',
    asset_tag: 'PMP-001',
    name: 'Test Pump',
    asset_type: 'pump',
    status: 'operational',
    criticality: 'high',
    health_index: 0.85,
    updated_at: new Date().toISOString(),
  }

  it('renders asset information correctly', () => {
    render(<AssetCard asset={mockAsset} />)

    expect(screen.getByText('Test Pump')).toBeInTheDocument()
    expect(screen.getByText('PMP-001')).toBeInTheDocument()
    expect(screen.getByText('operational')).toBeInTheDocument()
  })

  it('displays health index gauge', () => {
    render(<AssetCard asset={mockAsset} />)

    // Health gauge should show 85%
    expect(screen.getByTestId('health-gauge')).toBeInTheDocument()
  })

  it('calls onSelect when clicked', () => {
    const onSelect = vi.fn()
    render(<AssetCard asset={mockAsset} onSelect={onSelect} />)

    fireEvent.click(screen.getByRole('link'))

    expect(onSelect).toHaveBeenCalledWith(mockAsset)
  })

  it('shows correct status badge color', () => {
    const { rerender } = render(<AssetCard asset={mockAsset} />)

    // Operational = green
    expect(screen.getByText('operational')).toHaveClass('bg-green-500')

    // Degraded = yellow
    rerender(<AssetCard asset={{ ...mockAsset, status: 'degraded' }} />)
    expect(screen.getByText('degraded')).toHaveClass('bg-yellow-500')

    // Failed = red
    rerender(<AssetCard asset={{ ...mockAsset, status: 'failed' }} />)
    expect(screen.getByText('failed')).toHaveClass('bg-red-500')
  })
})
```

---

4. INTEGRATION TESTING

4.1 Database Integration Tests

```python
# tests/integration/test_asset_repository.py
import pytest
from uuid import uuid4
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

from app.models.base import Base
from app.models.asset import Asset
from app.models.location import Location
from app.repositories.asset_repository import AssetRepository
from app.schemas.asset import AssetCreate

@pytest.fixture(scope="module")
async def test_engine():
    """Create test database engine"""
    engine = create_async_engine(
        "postgresql+asyncpg://test:test@localhost:5432/oilgas_test",
        echo=True
    )
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

    yield engine

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)

    await engine.dispose()

@pytest.fixture
async def db_session(test_engine):
    """Create test database session"""
    async_session = sessionmaker(
        test_engine, class_=AsyncSession, expire_on_commit=False
    )
    async with async_session() as session:
        yield session
        await session.rollback()

@pytest.fixture
async def test_location(db_session):
    """Create test location"""
    location = Location(
        location_id=uuid4(),
        name="Test Location",
        location_type="unit"
    )
    db_session.add(location)
    await db_session.commit()
    return location
```

```python
class TestAssetRepository:
    """Integration tests for AssetRepository"""

    @pytest.mark.asyncio
    async def test_create_asset(self, db_session, test_location):
        """Test creating asset in database"""
        repo = AssetRepository(db_session)

        data = AssetCreate(
            asset_tag="INT-TEST-001",
            name="Integration Test Asset",
            asset_type="pump",
            location_id=test_location.location_id
        )

        asset = await repo.create(data)

        assert asset.asset_id is not None
        assert asset.asset_tag == "INT-TEST-001"

    @pytest.mark.asyncio
    async def test_get_by_tag(self, db_session, test_location):
        """Test retrieving asset by tag"""
        repo = AssetRepository(db_session)

        # Create asset
        data = AssetCreate(
            asset_tag="INT-TEST-002",
            name="Test Asset 2",
            asset_type="compressor",
            location_id=test_location.location_id
        )
        created = await repo.create(data)

        # Retrieve by tag
        asset = await repo.get_by_tag("INT-TEST-002")

        assert asset is not None
        assert asset.asset_id == created.asset_id

    @pytest.mark.asyncio
    async def test_get_by_type_and_status(self, db_session, test_location):
        """Test filtering assets by type and status"""
        repo = AssetRepository(db_session)

        # Create multiple assets
        for i in range(5):
            data = AssetCreate(
                asset_tag=f"INT-TEST-FILTER-{i}",
                name=f"Filter Test {i}",
                asset_type="pump" if i < 3 else "compressor",
                location_id=test_location.location_id
            )
```

```
            await repo.create(data)

        # Filter by type
        pumps, total = await repo.get_by_type_and_status(asset_type="pump")

        assert len(pumps) >= 3
        assert all(p.asset_type == "pump" for p in pumps)
```

4.2 API Integration Tests

```
# tests/integration/test_api_assets.py
import pytest
from httpx import AsyncClient
from uuid import uuid4

from app.main import app
from app.config import settings

@pytest.fixture
async def client():
    """Create test client"""
    async with AsyncClient(app=app, base_url="http://test") as ac:
        yield ac

@pytest.fixture
async def auth_token(client):
    """Get authentication token"""
    response = await client.post(
        "/v1/auth/token",
        data={
            "username": settings.test_username,
            "password": settings.test_password
        }
    )
    return response.json()["access_token"]

@pytest.fixture
def auth_headers(auth_token):
    """Create auth headers"""
    return {"Authorization": f"Bearer {auth_token}"}

class TestAssetAPI:
    """Integration tests for Asset API"""

    @pytest.mark.asyncio
    async def test_create_asset_endpoint(self, client, auth_headers):
        """Test POST /v1/assets"""
        response = await client.post(
            "/v1/assets",
            json={
                "asset_tag": f"API-TEST-{uuid4().hex[:8]}",
                "name": "API Test Asset",
                "asset_type": "pump",
                "location_id": "11111111-1111-1111-1111-111111111111",
```

```
            "criticality": "high"
        },
        headers=auth_headers
    )

    assert response.status_code == 201
    data = response.json()
    assert "asset_id" in data
    assert data["name"] == "API Test Asset"

@pytest.mark.asyncio
async def test_list_assets_endpoint(self, client, auth_headers):
    """Test GET /v1/assets"""
    response = await client.get(
        "/v1/assets",
        params={"page": 1, "limit": 10},
        headers=auth_headers
    )

    assert response.status_code == 200
    data = response.json()
    assert "data" in data
    assert "total" in data
    assert "page" in data

@pytest.mark.asyncio
async def test_get_asset_endpoint(self, client, auth_headers):
    """Test GET /v1/assets/{id}"""
    # First create an asset
    create_response = await client.post(
        "/v1/assets",
        json={
            "asset_tag": f"API-GET-{uuid4().hex[:8]}",
            "name": "Get Test Asset",
            "asset_type": "pump",
            "location_id": "11111111-1111-1111-1111-111111111111"
        },
        headers=auth_headers
    )
    asset_id = create_response.json()["asset_id"]

    # Then get it
    response = await client.get(
        f"/v1/assets/{asset_id}",
        headers=auth_headers
    )

    assert response.status_code == 200
    assert response.json()["asset_id"] == asset_id

@pytest.mark.asyncio
async def test_update_asset_endpoint(self, client, auth_headers):
    """Test PUT /v1/assets/{id}"""
    # Create
```

```
        create_response = await client.post(
            "/v1/assets",
            json={
                "asset_tag": f"API-UPD-{uuid4().hex[:8]}",
                "name": "Update Test Asset",
                "asset_type": "pump",
                "location_id": "11111111-1111-1111-1111-111111111111"
            },
            headers=auth_headers
        )
        asset_id = create_response.json()["asset_id"]

        # Update
        response = await client.put(
            f"/v1/assets/{asset_id}",
            json={"name": "Updated Name"},
            headers=auth_headers
        )

        assert response.status_code == 200
        assert response.json()["name"] == "Updated Name"

    @pytest.mark.asyncio
    async def test_delete_asset_endpoint(self, client, auth_headers):
        """Test DELETE /v1/assets/{id}"""
        # Create
        create_response = await client.post(
            "/v1/assets",
            json={
                "asset_tag": f"API-DEL-{uuid4().hex[:8]}",
                "name": "Delete Test Asset",
                "asset_type": "pump",
                "location_id": "11111111-1111-1111-1111-111111111111"
            },
            headers=auth_headers
        )
        asset_id = create_response.json()["asset_id"]

        # Delete
        response = await client.delete(
            f"/v1/assets/{asset_id}",
            headers=auth_headers
        )

        assert response.status_code == 204

        # Verify deleted
        get_response = await client.get(
            f"/v1/assets/{asset_id}",
            headers=auth_headers
        )
        assert get_response.status_code == 404
```

5. API TESTING

5.1 Postman Collection Structure

```
{
  "info": {
    "name": "Oil & Gas AI Platform API",
    "description": "API test collection for Oil & Gas AI monitoring platform"
  },
  "auth": {
    "type": "bearer",
    "bearer": [
      {
        "key": "token",
        "value": "{{access_token}}"
      }
    ]
  },
  "variable": [
    {
      "key": "base_url",
      "value": "https://api.oilgas.internal"
    },
    {
      "key": "access_token",
      "value": ""
    }
  ],
  "item": [
    {
      "name": "Authentication",
      "item": [
        {
          "name": "Get Access Token",
          "request": {
            "method": "POST",
            "url": "{{base_url}}/v1/auth/token"
          }
        }
      ]
    },
    {
      "name": "Assets",
      "item": [
        {
          "name": "List Assets",
          "request": {
            "method": "GET",
            "url": "{{base_url}}/v1/assets"
          }
        },
        {
          "name": "Create Asset",
          "request": {
```

```
            "method": "POST",
            "url": "{{base_url}}/v1/assets"
          }
        }
      ]
    }
  ]
}
```

5.2 API Test Cases

Test ID	Endpoint	Method	Test Case	Expected
API-001	/v1/auth/token	POST	Valid credentials	200 + token
API-002	/v1/auth/token	POST	Invalid credentials	401
API-003	/v1/assets	GET	List all assets	200 + array
API-004	/v1/assets	GET	Filter by type	200 + filtered
API-005	/v1/assets	GET	Pagination	200 + paginated
API-006	/v1/assets	POST	Valid payload	201 + created
API-007	/v1/assets	POST	Missing required field	422
API-008	/v1/assets	POST	Duplicate tag	409
API-009	/v1/assets/{id}	GET	Existing asset	200 + asset
API-010	/v1/assets/{id}	GET	Non-existent	404
API-011	/v1/assets/{id}	PUT	Valid update	200 + updated
API-012	/v1/assets/{id}	DELETE	Existing asset	204
API-013	/v1/rag/query	POST	Valid query	200 + response
API-014	/v1/rag/query	POST	Empty query	422
API-015	/v1/predictions/{id}	GET	Asset predictions	200 + array

---

6. PERFORMANCE TESTING

6.1 Load Testing with Locust

```python
# tests/performance/locustfile.py
from locust import HttpUser, task, between
import random

class APIUser(HttpUser):
    """Simulates API user behavior"""
    wait_time = between(1, 3)
    token = None

    def on_start(self):
        """Login and get token"""
        response = self.client.post(
            "/v1/auth/token",
            data={
                "username": "load_test_user",
                "password": "load_test_pass"
            }
        )
        self.token = response.json()["access_token"]
```

```
        self.headers = {"Authorization": f"Bearer {self.token}"}

    @task(10)
    def list_assets(self):
        """List assets (most common operation)"""
        self.client.get(
            "/v1/assets",
            headers=self.headers,
            params={"page": 1, "limit": 20}
        )

    @task(5)
    def get_asset(self):
        """Get specific asset"""
        asset_ids = [
            "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
            "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb"
        ]
        self.client.get(
            f"/v1/assets/{random.choice(asset_ids)}",
            headers=self.headers
        )

    @task(3)
    def rag_query(self):
        """RAG query"""
        queries = [
            "What is the maintenance procedure for centrifugal pumps?",
            "How to troubleshoot high vibration in compressors?",
            "What are the safety procedures for gas leak detection?"
        ]
        self.client.post(
            "/v1/rag/query",
            headers=self.headers,
            json={"query": random.choice(queries)}
        )

    @task(1)
    def get_predictions(self):
        """Get asset predictions"""
        self.client.get(
            "/v1/assets/aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa/predictions",
            headers=self.headers
        )

class DashboardUser(HttpUser):
    """Simulates dashboard user"""
    wait_time = between(5, 15)
    token = None

    def on_start(self):
        response = self.client.post(
            "/v1/auth/token",
            data={"username": "dashboard_user", "password": "dashboard_pass"}
```

```
        )
        self.token = response.json()["access_token"]
        self.headers = {"Authorization": f"Bearer {self.token}"}

    @task
    def dashboard_load(self):
        """Simulate dashboard page load"""
        # Assets summary
        self.client.get(
            "/v1/assets",
            headers=self.headers,
            params={"limit": 50}
        )

        # Active alerts
        self.client.get(
            "/v1/alerts",
            headers=self.headers,
            params={"status": "active", "limit": 10}
        )

        # Recent predictions
        self.client.get(
            "/v1/predictions/recent",
            headers=self.headers,
            params={"limit": 20}
        )
```

Running Locust

```
# Start Locust web UI
locust -f tests/performance/locustfile.py --host=https://api.oilgas.internal

# Run headless
locust -f tests/performance/locustfile.py \
  --host=https://api.oilgas.internal \
  --headless \
  --users 100 \
  --spawn-rate 10 \
  --run-time 10m \
  --csv=reports/load_test

# Distributed load testing
# Master
locust -f locustfile.py --master

# Workers (run on multiple machines)
locust -f locustfile.py --worker --master-host=<master-ip>
```

6.2 Performance Benchmarks

Metric	Target	Critical
API Response Time (P50)	<200ms	<500ms

Metric	Target	Critical
API Response Time (P95)	<500ms	<1000ms
API Response Time (P99)	<1000ms	<2000ms
Throughput	>1000 RPS	>500 RPS
Error Rate	<0.1%	<1%
RAG Query Time	<3s	<5s
ML Inference Time	<100ms	<200ms
Database Query Time	<50ms	<100ms

6.3 Performance Test Report Template

```
# Performance Test Report

**Test Date**: 2025-12-10
**Environment**: QA
**Test Duration**: 60 minutes
**Users**: 100 concurrent

## Summary

| Metric | Result | Target | Status |
|--------|--------|--------|--------|
| P50 Response Time | 180ms | <200ms | PASS |
| P95 Response Time | 420ms | <500ms | PASS |
| P99 Response Time | 850ms | <1000ms | PASS |
| Throughput | 1,250 RPS | >1000 RPS | PASS |
| Error Rate | 0.05% | <0.1% | PASS |

## Endpoint Performance

| Endpoint | P50 | P95 | RPS | Errors |
|----------|-----|-----|-----|--------|
| GET /v1/assets | 120ms | 280ms | 450 | 0 |
| GET /v1/assets/{id} | 85ms | 180ms | 300 | 0 |
| POST /v1/rag/query | 2.1s | 3.8s | 50 | 2 |
| GET /v1/predictions | 95ms | 220ms | 200 | 0 |

## Resource Utilization

| Resource | Average | Peak |
|----------|---------|------|
| CPU (API pods) | 45% | 72% |
| Memory (API pods) | 2.1GB | 2.8GB |
| Database CPU | 35% | 58% |
| Database Connections | 45 | 78 |

## Recommendations

1. Consider caching RAG responses for common queries
2. Database connection pool can be increased
3. API pods can be scaled horizontally at 70% CPU
```

7. SECURITY TESTING

7.1 OWASP Top 10 Checklist

#	Vulnerability	Test Method	Status
A01	Broken Access Control	Burp Suite, manual testing	-
A02	Cryptographic Failures	SSL Labs, cipher analysis	-
A03	Injection	SQLMap, manual payloads	-
A04	Insecure Design	Architecture review	-
A05	Security Misconfiguration	Nikto, config review	-
A06	Vulnerable Components	OWASP Dependency Check	-
A07	Authentication Failures	Burp Suite, brute force	-
A08	Data Integrity Failures	Deserialization tests	-
A09	Logging Failures	Log review	-
A10	SSRF	Manual testing	-

7.2 Security Test Cases

```python
# tests/security/test_authentication.py
import pytest
from httpx import AsyncClient

class TestAuthentication:
    """Security tests for authentication"""

    @pytest.mark.asyncio
    async def test_sql_injection_login(self, client):
        """Test SQL injection in login"""
        payloads = [
            "' OR '1'='1",
            "admin'--",
            "1; DROP TABLE users--"
        ]

        for payload in payloads:
            response = await client.post(
                "/v1/auth/token",
                data={"username": payload, "password": "test"}
            )
            # Should return 401, not 500 or successful login
            assert response.status_code in [401, 422]

    @pytest.mark.asyncio
    async def test_brute_force_protection(self, client):
        """Test account lockout after failed attempts"""
        for i in range(10):
            await client.post(
                "/v1/auth/token",
                data={"username": "test@oilgas.com", "password": "wrong"}
            )

        # 11th attempt should be blocked
        response = await client.post(
```

```
            "/v1/auth/token",
            data={"username": "test@oilgas.com", "password": "wrong"}
        )
        assert response.status_code == 429  # Too Many Requests

    @pytest.mark.asyncio
    async def test_jwt_manipulation(self, client, valid_token):
        """Test JWT token tampering"""
        # Modify token payload
        parts = valid_token.split('.')
        modified_token = parts[0] + '.' + 'invalid_payload' + '.' + parts[2]

        response = await client.get(
            "/v1/assets",
            headers={"Authorization": f"Bearer {modified_token}"}
        )
        assert response.status_code == 401

    @pytest.mark.asyncio
    async def test_expired_token(self, client, expired_token):
        """Test expired JWT token rejection"""
        response = await client.get(
            "/v1/assets",
            headers={"Authorization": f"Bearer {expired_token}"}
        )
        assert response.status_code == 401
```

7.3 Container Security Scan

```
# Scan Docker images with Trivy
trivy image oilgas/api:latest --severity HIGH,CRITICAL

# Example output
oilgas/api:latest (ubuntu 22.04)
==================================
Total: 0 (HIGH: 0, CRITICAL: 0)

Python (pip)
============
Total: 2 (HIGH: 1, CRITICAL: 1)

+-----------------+------------------+----------+-------------------+---------------+
|     Library     |  Vulnerability   | Severity | Installed Version | Fixed Version |
+-----------------+------------------+----------+-------------------+---------------+
| cryptography    | CVE-2023-XXXXX   | CRITICAL | 41.0.0            | 41.0.2        |
| requests        | CVE-2023-YYYYY   | HIGH     | 2.28.0            | 2.31.0        |
+-----------------+------------------+----------+-------------------+---------------+
```

8. ML MODEL TESTING

8.1 Model Validation Tests

```python
# tests/ml/test_anomaly_model.py
import pytest
import numpy as np
import torch
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score

from ml_models.anomaly_detection.src.model import AnomalyAutoencoder
from ml_models.anomaly_detection.src.preprocess import normalize_features

class TestAnomalyModel:
    """Tests for anomaly detection model"""

    @pytest.fixture
    def model(self):
        model = AnomalyAutoencoder(
            input_dim=15,
            latent_dim=3,
            hidden_dims=[10, 5]
        )
        model.load_state_dict(torch.load("models/anomaly_detector.pt"))
        model.eval()
        return model

    @pytest.fixture
    def test_data(self):
        """Load test dataset"""
        return np.load("tests/data/anomaly_test_data.npz")

    def test_model_accuracy(self, model, test_data):
        """Test model achieves minimum accuracy"""
        X_test = test_data["X"]
        y_test = test_data["y"]

        with torch.no_grad():
            X_tensor = torch.FloatTensor(X_test)
            reconstructed, _ = model(X_tensor)
            errors = torch.mean((X_tensor - reconstructed) ** 2, dim=1).numpy()

        # Use calibrated threshold
        threshold = 0.05  # From training
        predictions = (errors > threshold).astype(int)

        accuracy = np.mean(predictions == y_test)
        assert accuracy >= 0.95, f"Accuracy {accuracy:.3f} below 95% threshold"

    def test_model_precision_recall(self, model, test_data):
        """Test precision and recall meet requirements"""
        X_test = test_data["X"]
        y_test = test_data["y"]
```

```
        with torch.no_grad():
            X_tensor = torch.FloatTensor(X_test)
            reconstructed, _ = model(X_tensor)
            errors = torch.mean((X_tensor - reconstructed) ** 2, dim=1).numpy()

        threshold = 0.05
        predictions = (errors > threshold).astype(int)

        precision = precision_score(y_test, predictions)
        recall = recall_score(y_test, predictions)
        f1 = f1_score(y_test, predictions)

        # Requirements
        assert precision >= 0.90, f"Precision {precision:.3f} below 90%"
        assert recall >= 0.85, f"Recall {recall:.3f} below 85%"
        assert f1 >= 0.87, f"F1 {f1:.3f} below 87%"

    def test_model_auc_roc(self, model, test_data):
        """Test AUC-ROC score"""
        X_test = test_data["X"]
        y_test = test_data["y"]

        with torch.no_grad():
            X_tensor = torch.FloatTensor(X_test)
            reconstructed, _ = model(X_tensor)
            errors = torch.mean((X_tensor - reconstructed) ** 2, dim=1).numpy()

        auc = roc_auc_score(y_test, errors)
        assert auc >= 0.95, f"AUC-ROC {auc:.3f} below 95%"

    def test_false_positive_rate(self, model, test_data):
        """Test false positive rate is acceptable"""
        X_normal = test_data["X_normal"]  # Only normal samples

        with torch.no_grad():
            X_tensor = torch.FloatTensor(X_normal)
            reconstructed, _ = model(X_tensor)
            errors = torch.mean((X_tensor - reconstructed) ** 2, dim=1).numpy()

        threshold = 0.05
        false_positives = np.mean(errors > threshold)

        assert false_positives <= 0.05, f"FP rate {false_positives:.3f} exceeds 5%"

    def test_inference_latency(self, model):
        """Test inference time meets requirements"""
        import time

        X_batch = torch.randn(100, 15)
        model.eval()

        # Warmup
        with torch.no_grad():
            _ = model(X_batch)
```

```
        # Measure
        start = time.time()
        with torch.no_grad():
            for _ in range(100):
                _ = model(X_batch)
        elapsed = (time.time() - start) / 100

        latency_ms = elapsed * 1000
        assert latency_ms < 10, f"Inference {latency_ms:.1f}ms exceeds 10ms target"

    def test_model_robustness(self, model):
        """Test model handles edge cases"""
        # Zero input
        X_zeros = torch.zeros(1, 15)
        with torch.no_grad():
            out, _ = model(X_zeros)
        assert not torch.isnan(out).any()

        # Very large values
        X_large = torch.ones(1, 15) * 1000
        with torch.no_grad():
            out, _ = model(X_large)
        assert not torch.isnan(out).any()

        # Negative values
        X_neg = torch.ones(1, 15) * -10
        with torch.no_grad():
            out, _ = model(X_neg)
        assert not torch.isnan(out).any()
```

8.2 Model Performance Benchmarks

Model	Metric	Target	Current	Status
Anomaly Detection	Accuracy	>95%	96.2%	PASS
Anomaly Detection	Precision	>90%	92.1%	PASS
Anomaly Detection	Recall	>85%	88.5%	PASS
Anomaly Detection	F1-Score	>87%	90.2%	PASS
Anomaly Detection	AUC-ROC	>95%	97.3%	PASS
Anomaly Detection	FP Rate	<5%	3.2%	PASS
Anomaly Detection	Latency	<10ms	2.3ms	PASS
RUL Prediction	RMSE	<15 days	12.4 days	PASS
RUL Prediction	MAE	<10 days	8.7 days	PASS
PPE Detection	mAP@50	>85%	89.2%	PASS
Corrosion Detection	Accuracy	>90%	91.5%	PASS

9. EDGE DEVICE TESTING

9.1 Edge Integration Tests

```python
# tests/edge/test_edge_integration.py
import pytest
import requests
import time

class TestEdgeDeviceIntegration:
    """Integration tests for edge devices"""

    @pytest.fixture
    def edge_endpoint(self):
        return "http://edge-test-01:8080"

    def test_edge_health_check(self, edge_endpoint):
        """Test edge device health endpoint"""
        response = requests.get(f"{edge_endpoint}/health", timeout=5)

        assert response.status_code == 200
        health = response.json()
        assert health["status"] == "healthy"
        assert "gpu_available" in health
        assert "disk_space_gb" in health

    def test_edge_inference(self, edge_endpoint):
        """Test edge inference endpoint"""
        # Generate test vibration data
        test_data = {
            "sensor_id": "TEST-VIB-001",
            "waveform": [0.1] * 8192,  # 8192 samples
            "sampling_rate": 25600,
            "rpm": 3000
        }

        response = requests.post(
            f"{edge_endpoint}/inference",
            json=test_data,
            timeout=5
        )

        assert response.status_code == 200
        result = response.json()
        assert "anomaly_score" in result
        assert "health_index" in result
        assert 0 <= result["health_index"] <= 1

    def test_edge_inference_latency(self, edge_endpoint):
        """Test inference latency meets requirements"""
        test_data = {
            "sensor_id": "TEST-VIB-001",
            "waveform": [0.1] * 8192,
            "sampling_rate": 25600,
```

```
        "rpm": 3000
    }

    latencies = []
    for _ in range(10):
        start = time.time()
        response = requests.post(
            f"{edge_endpoint}/inference",
            json=test_data,
            timeout=5
        )
        latency_ms = (time.time() - start) * 1000
        latencies.append(latency_ms)

    avg_latency = sum(latencies) / len(latencies)
    p95_latency = sorted(latencies)[int(len(latencies) * 0.95)]

    assert avg_latency < 100, f"Avg latency {avg_latency:.1f}ms exceeds 100ms"
    assert p95_latency < 200, f"P95 latency {p95_latency:.1f}ms exceeds 200ms"

def test_edge_video_processing(self, edge_endpoint):
    """Test video frame processing"""
    # Read test image
    with open("tests/data/test_frame.jpg", "rb") as f:
        image_data = f.read()

    response = requests.post(
        f"{edge_endpoint}/cv/detect",
        files={"image": ("frame.jpg", image_data, "image/jpeg")},
        timeout=10
    )

    assert response.status_code == 200
    result = response.json()
    assert "detections" in result
    assert "processing_time_ms" in result
    assert result["processing_time_ms"] < 100

def test_edge_offline_mode(self, edge_endpoint):
    """Test edge device continues working when cloud is unavailable"""
    # Simulate cloud disconnect
    requests.post(f"{edge_endpoint}/debug/disconnect_cloud")

    # Inference should still work
    test_data = {
        "sensor_id": "TEST-VIB-001",
        "waveform": [0.1] * 8192,
        "sampling_rate": 25600,
        "rpm": 3000
    }

    response = requests.post(
        f"{edge_endpoint}/inference",
        json=test_data,
```

```
    timeout=5
)

assert response.status_code == 200

# Reconnect
requests.post(f"{edge_endpoint}/debug/connect_cloud")
```

9.2 Edge Device Test Matrix

Test	Jetson AGX	Jetson Nano	Advantech	Status
Boot time	<60s	<90s	<45s	-
Health endpoint	<50ms	<100ms	<50ms	-
Vibration inference	<50ms	<200ms	N/A	-
Video processing	<100ms	<300ms	N/A	-
Concurrent streams	4	1	0	-
Offline operation	24h	24h	24h	-
Memory usage	<8GB	<3GB	<2GB	-
GPU utilization	<70%	<85%	N/A	-

---

10. USER ACCEPTANCE TESTING

10.1 UAT Test Scenarios

Scenario 1: Asset Health Monitoring

Step	Action	Expected Result
1	Login as Operator	Dashboard loads
2	View asset list	All assets visible with status
3	Click on degraded asset	Asset detail page shows
4	View sensor readings	Current values and trends display
5	Check health index	Health gauge shows correct value
6	View predictions	RUL prediction visible
7	Create maintenance ticket	Ticket created in SAP

Scenario 2: Alert Management

Step	Action	Expected Result
1	Receive alert notification	Email/SMS received
2	View alert in dashboard	Alert details show
3	Acknowledge alert	Status changes to “Acknowledged”
4	Add comment	Comment saved
5	Resolve alert	Status changes to “Resolved”
6	View alert history	All alerts listed

Scenario 3: RAG Knowledge Query

Step	Action	Expected Result
1	Open chat interface	Chat ready
2	Ask: “How to calibrate pump P-101?”	Relevant answer with sources
3	Ask follow-up question	Context maintained
4	Click source reference	Document opens
5	Rate response	Feedback recorded

10.2 UAT Sign-off Template

```
# UAT Sign-off Document

**Project**: Oil & Gas AI Monitoring Platform
**Release**: v1.0.0
**UAT Period**: 2025-12-01 to 2025-12-15

## Test Summary

| Category | Total | Passed | Failed | Blocked |
|----------|-------|--------|--------|---------|
| Asset Monitoring | 15 | 14 | 1 | 0 |
| Alert Management | 12 | 12 | 0 | 0 |
| RAG Queries | 10 | 9 | 0 | 1 |
| Predictions | 8 | 8 | 0 | 0 |
| Reports | 5 | 5 | 0 | 0 |
| **Total** | **50** | **48** | **1** | **1** |

## Outstanding Issues

| ID | Description | Severity | Workaround |
|----|-------------|----------|------------|
| UAT-001 | Export to Excel truncates long names | Low | Use PDF export |
| UAT-002 | RAG slow for complex queries | Medium | Break into simpler queries |

## Sign-off

I confirm that the system has been tested against the agreed acceptance criteria and is ready for produ

| Role | Name | Signature | Date |
|------|------|-----------|------|
| Business Owner | | | |
| Operations Lead | | | |
| IT Manager | | | |
| QA Lead | | | |
```

---

11. TEST AUTOMATION

11.1 CI/CD Test Pipeline

```yaml
# .github/workflows/test.yml
name: Test Pipeline
```

```yaml
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Run unit tests
        run: |
          pytest tests/unit/ \
            --cov=app \
            --cov-report=xml \
            --junitxml=reports/unit-tests.xml

      - name: Upload coverage
        uses: codecov/codecov-action@v3

  integration-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v4

      - name: Run integration tests
        run: |
          pytest tests/integration/ \
            --junitxml=reports/integration-tests.xml

  api-tests:
    runs-on: ubuntu-latest
```

```yaml
    needs: integration-tests
    steps:
      - uses: actions/checkout@v4

      - name: Run API tests
        run: |
          pytest tests/api/ \
            --junitxml=reports/api-tests.xml

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          severity: 'HIGH,CRITICAL'

      - name: Run Bandit
        run: |
          pip install bandit
          bandit -r app/ -f json -o reports/bandit.json

  publish-results:
    runs-on: ubuntu-latest
    needs: [unit-tests, integration-tests, api-tests]
    steps:
      - name: Publish test results
        uses: dorny/test-reporter@v1
        with:
          name: Test Results
          path: 'reports/*.xml'
          reporter: java-junit
```

11.2 Nightly Test Schedule

```yaml
# .github/workflows/nightly.yml
name: Nightly Tests

on:
  schedule:
    - cron: '0 2 * * *'  # 2 AM daily

jobs:
  full-regression:
    runs-on: ubuntu-latest
    steps:
      - name: Run full regression suite
        run: pytest tests/ --junitxml=reports/nightly.xml

  performance-test:
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Run performance tests
        run: |
          locust -f tests/performance/locustfile.py \
            --headless \
            --users 50 \
            --spawn-rate 5 \
            --run-time 30m \
            --csv=reports/perf

  ml-validation:
    runs-on: ubuntu-latest
    steps:
      - name: Validate ML models
        run: pytest tests/ml/ --junitxml=reports/ml-tests.xml
```

---

12. DEFECT MANAGEMENT

12.1 Defect Severity Levels

Level	Name	Description	SLA
S1	Critical	System down, data loss, security breach	4 hours
S2	High	Major feature broken, no workaround	1 day
S3	Medium	Feature impaired, workaround exists	3 days
S4	Low	Minor issue, cosmetic	1 week

12.2 Defect Report Template

```
## Defect Report

**ID**: DEF-2025-001
**Title**: API returns 500 error when fetching asset with special characters
**Severity**: S2 - High
**Priority**: P1
**Reporter**: QA Team
**Date**: 2025-12-10

### Environment
- Environment: QA
- Version: 1.0.0-rc1
- Browser: N/A (API)

### Steps to Reproduce
1. Create asset with name containing special characters: "Pump (A/B) #1"
2. Call GET /v1/assets/{id}
3. Observe response

### Expected Result
Asset details returned with 200 OK

### Actual Result
```

```
500 Internal Server Error
```json
{
  "detail": "Internal server error"
}
```

Logs

```
2025-12-10 10:23:45 ERROR - UnicodeDecodeError: 'utf-8' codec can't decode byte...
```

Root Cause

Database encoding mismatch

Fix

PR #123 - Update database connection to use UTF-8

Verification

- ☐ Unit test added
- ☐ Integration test added
- ☐ Tested on QA
- ☐ Tested on Staging “‘

12.3 Defect Metrics Dashboard

Metric	Current	Target
Open defects (S1+S2)	2	0
Open defects (total)	15	<20
Defect resolution time (avg)	2.3 days	<3 days
Defects found in production	1	0
Test coverage	82%	>80%
Pass rate	96%	>95%

END OF TESTING GUIDE
```